Unit 2: Database System – Concepts and Architecture (3 Hrs.)

Data Models, Schemas, and Instances; Three-Schema Architecture and Data Independence; Database Languages and Interfaces; the Database System Environment; Centralized and Client/Server Architectures for DBMSs; Classification of Database Management Systems

Data Model

Data model is a way organizing a collection of information pertaining to a system under investigation.

A database model is a collection of conceptual tools for describing data relationship, data semantics, and consistency constraints.

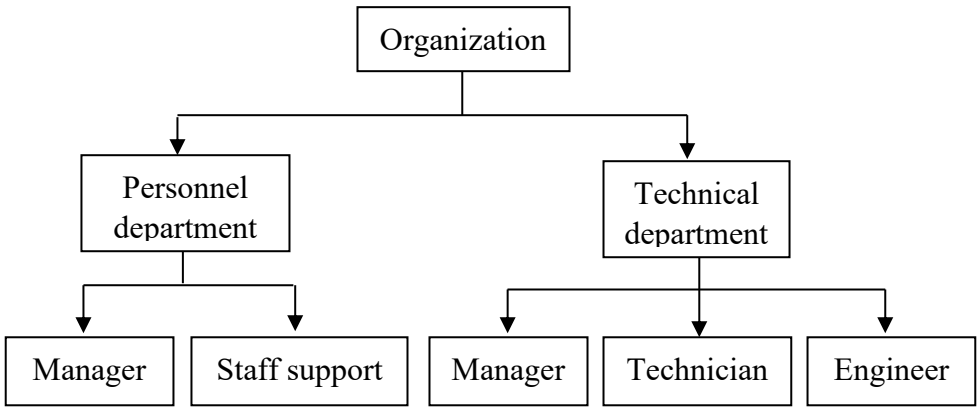
It consists of two parts:

1) A mathematical notation for describing the data and relationship.
2) A set of operation to manipulate the data.

Every database and database management system is based on a particular database model. ***A database model consists of rules and standards that define how data is organized in database.***

It provides strong theoretical foundation of database structures. There are four types of database model.

1. **Hierarchical Database Model:**
   - In hierarchical data model the data elements are liked in the inverted tree structure with the root at the top and branches formed below.
   - There is a parent-child relationship among the data elements of a hierarchical data base.
   - There are many child elements under each parent elements, but there can be only one parent element for any child element. The branches in the tree are not connected.



i) It is the easiest model of data base.

ii) A data base owner is more secure because nobody else can see and modify by a child without consulting its parents.

iii) Searching is fast and easy, if parent is known.

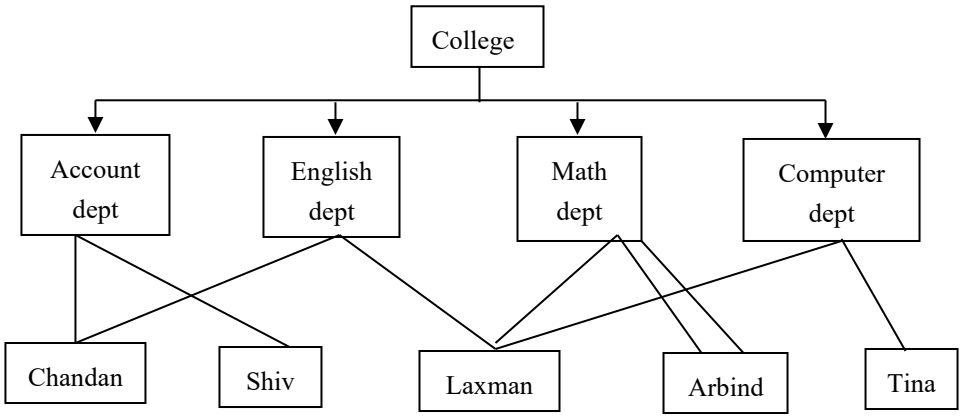
iv) It is very efficient in handling "one-to-many" relationship.

Disadvantages

- It is old fashioned outdated data base model.
- It cannot handle "many-to-many" relationship.
- It increases data redundancy as same data can be saved in many places.

2. Networking Database Model:

- A networking database model is an extension of the hierarchical database structures.
- In this model also the data elements of a database are organized in the form of parent child relationships among the data elements.
- In the network data base, a child data element can have more than one parent element or no parent at all. i.e. it's support many-to-many relationship.



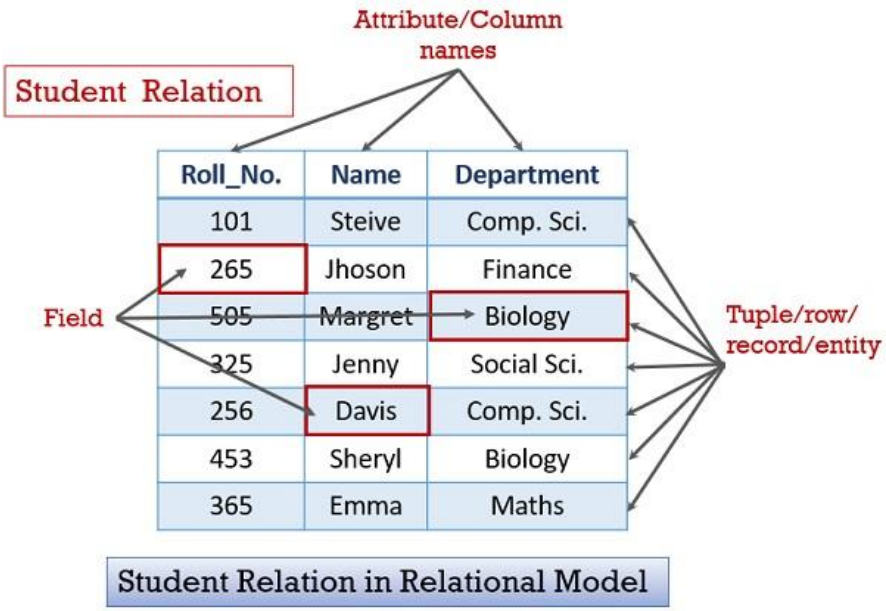
Advantages

- More flexible than hierarchical data base model because it accepts many-to-many relationship.
- Searching is faster because of multi directional pointer.
- It reduces redundancy because data should not be repeated if data is needed.

Disadvantages

- Less secure than hierarchical as it is open to all.
- Need long program to handle the relation ship
- It is one of the complex data model.

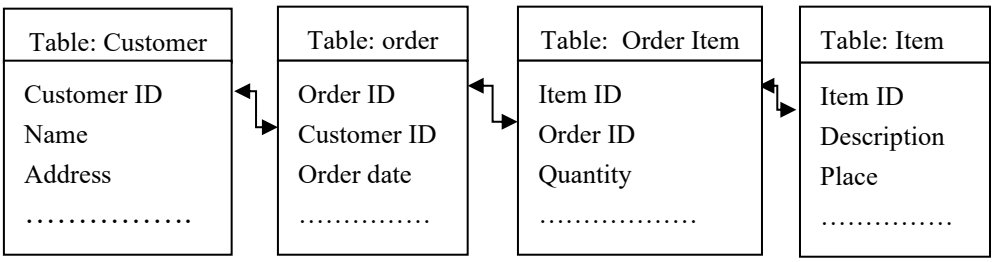
3. Relation Database Model:



- In this model, data is organized in the form of table with rows and columns.
- Each table of database is saved as a separate file each table column represents a data filed and each row represents a data record.
- A relation is also known as table. The data in one table is related to data in another table with the common filed.

The relational data base model provides greater flexibility of data organization and future enhancement in data base as compare to hierarchical and network data base model.

If new data is to be added an existing relational data base, it is not necessary to reclosing the data base. Rather new table containing the new data can be added to the data base and this table can easily be implemented in another.



Advantages

- Since one table is liked with other, same common fields field rule implemented on one table can be easily implemented to another.
- It has very less data redundancy.
- Qaida data base processing is possible.

- Normalization of data base is possible.

Disadvantages

- It is more complex than other models.
- Too many rules make data base less user friendly.

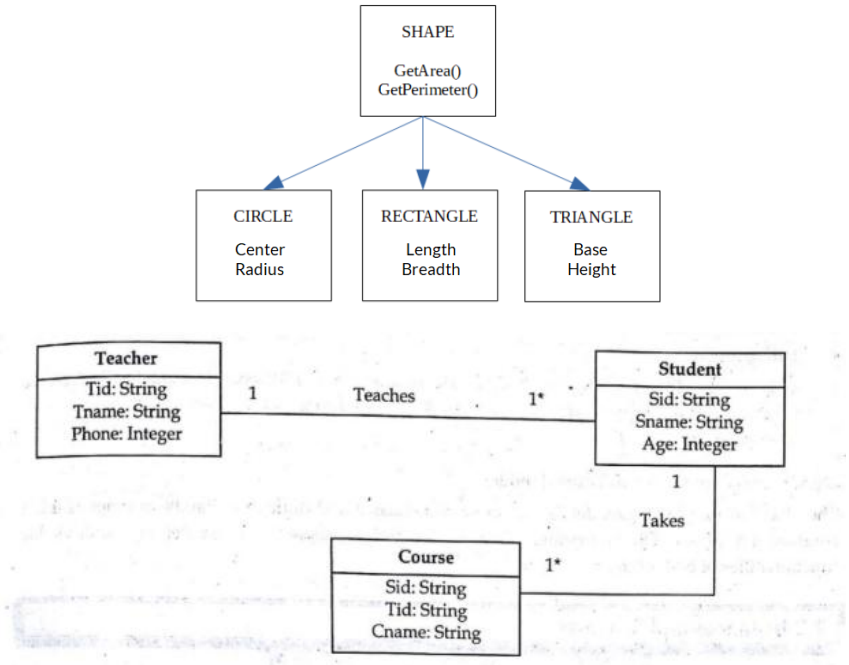
4. Object Oriented Database:

- The Object-Oriented Data Model (OODM) is a database approach that integrates concepts from object-oriented programming with traditional data modeling, allowing data and its relationships to be represented as objects with attributes and methods.
- It is especially useful for modeling complex, real-world entities like multimedia, engineering designs, or scientific data.

Concepts of OODM

Objects: Represent real-world entities (e.g., Student, Doctor, Car). Each object encapsulates both data (attributes) and behavior (methods).

Attributes : Properties that describe an object. Example: A Student object may have attributes like



Roll Number, Name, and Branch.

Methods : Define the behavior of objects. Example: A Student object may have methods like calculateMarks() or updateProfile().

Classes : Collections of similar objects. A class defines the structure (attributes) and behavior (methods) shared by its objects.

Inheritance : Mechanism where new classes derive attributes and methods from existing ones. Example: Student, Doctor, and Engineer classes may inherit from a Person class.

Advantages

- It excels at representing real-world complexity, integrates seamlessly with OOP languages, and reduces data duplication through reuse.
- This model supports multimedia data (e.g., images, videos) better than traditional relational approaches.

Applications

- OODMs power object-oriented database management systems (OODBMS) in fields like e-commerce and CAD, where entity relationships are intricate

Disadvantages

i) It is most complex form of database model.
ii) To create this model concept of class, objects, attributes, etc must be defined.

5. E-R data model

- An Entity–relationship model (ER model) describes the structure of a database with the help of a diagram, which is known as Entity Relationship Diagram (ER Diagram).
- An ER model is a design or blueprint of a database that can later be implemented as a database.
- The main components of E-R model are: entity set and relationship set.

Components of ER Model:

1. Entity: An entity is referred to as a real-world object. It can be a name, place, object, class, etc. These are represented by a rectangle in an ER Diagram.
2. Attributes: An attribute can be defined as the description of the entity. These are represented by Ellipse in an ER Diagram. It can be Age, Roll Number, or Marks for a Student.
3. Relationship: Relationships are used to define relations among different entities. Diamonds and Rhombus are used to show Relationships.

- **Database Schema:** A database schema is description of database simply the database schema is the logical structure of the database for example the database consists of information about a set of customers and accounts and the relation between them.

The data base systems have several schemas and partitioned according to the level of abstraction such as physical and logical schema.

- **Instance:** The collection of information stored in the database at a particular moment is called an instance of data base.

Database Architecture: The DBMS architecture describes how data in the database is viewed by users. It is not concerned with how the data is handled and processed by DBMS.

The database users are provided with an abstract view of data by hiding certain details of how data is physically store. This enables the users to manipulate the data without worrying about where it is located or how it is actually stored.

If this architectures the overall data base description can be defined at three levels.

A. Internal Level (Physical Level)

- It is the lowest level of data abstraction that deals with the physical representation of database on the computer and thus, is also known as physical level.
- It describes how the data is physically store and organized on the storage medium.
- At this level various aspects such as indexes, data compression and encryption technique etc are consider to achieve optimal run time performance and storage space utilization.

B. Conceptual Level

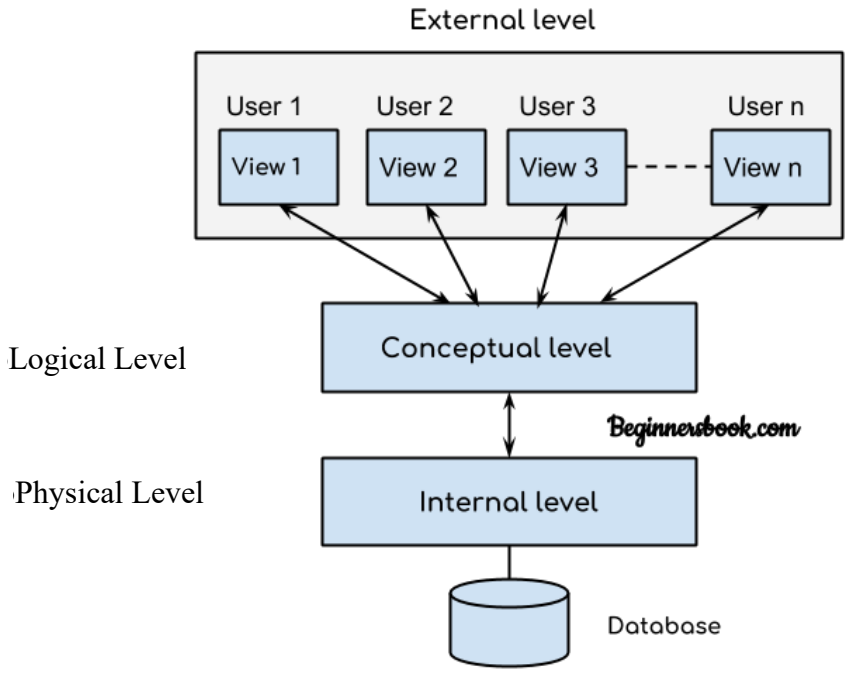
- This level of abstraction describes what data are actually store in data base.
- It also describes the relationship existing among data. Here, the database is described logically in term of simple data structures.
- It describe the relationship among the data and complete view of user's requirement without any concern for the physical implementation i.e. it hides the complexity of physical storage structure.

C. External Level (view Level)

This is the level closest to the user and is concerned with the way in which the data are viewed by individual user most of the users of the data base are not concerned with all the information contained in the data base. Instead they need only a part of the data base to them.

For Example

Even though the bank database stores a lot many information, an account holder if interested only in his account details is and not with the rest of the information stored in the data base.

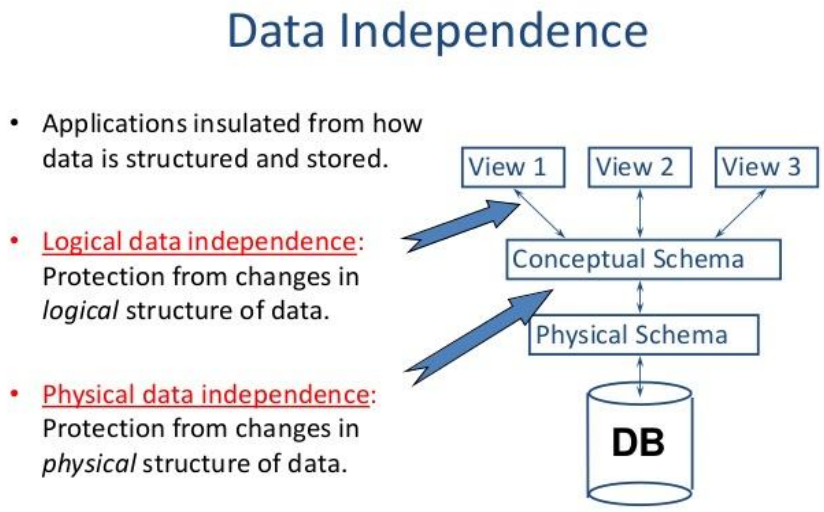


.

Data Independence:

The ability to modify a schema definition in one level without affecting a schema definition in the higher level is called data independence. Data independence is one of the main advantages of DBMS. It is of two types:-

i. Physical data Independence Logical data Independence



i) Physical Data Independence

- It refers to the ability to modify the schema followed at the physical level without affecting the schema allowed at the conceptual level.
- That is the application program remains the same even though the schema at physical level gets modified.
- Modification at the physical level is occasionally necessary in order to improve performance
- Example of physical independence is re-organization of file adding a new access by etc.

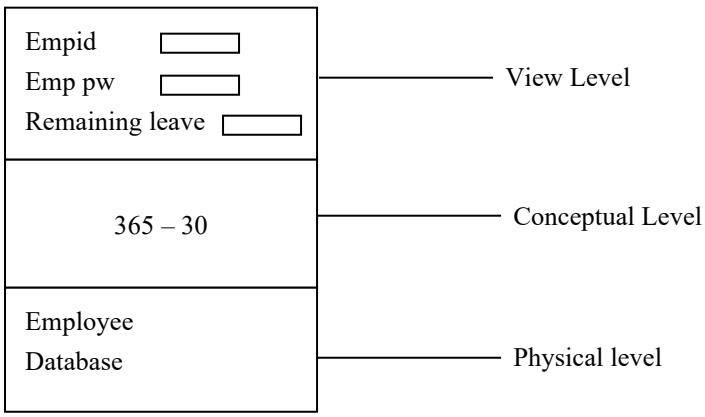
ii) Logical Data Independence

- It refers to the ability to modify the conceptual schema without causing any changes in the schema followed at view level

  The logical data independence answers that the program remains the same.
- Modification at the conceptual level are necessary whenever logical structures of data base get altered because of some unavoidable reason.

Example

The introduction of paternity leave in employed data base at the first time.



Database Languages

A DBMS must provide appropriate languages and interfaces for each category of users to express database queries and updates. Database Languages are used to create and maintain database on computer. There are large numbers of database languages like Oracle, MySQL, MS Access, dBase, FoxPro etc. SQL statements commonly used in Oracle and MS Access can be categorized as data definition language (DDL), data control language (DCL) and data manipulation language (DML).

Data Definition Language (DDL)

It is a language that allows the users to define data and their relationship to other types of data. It is mainly used to create files, databases, data dictionary and tables within databases.

It is also used to specify the structure of each table, set of associated values with each attribute, integrity constraints, security and authorization information for each table and physical storage structure of each table on disk.

- **CREATE** - to create objects in the database
- **ALTER** - alters the structure of the database
- **DROP** - delete objects from the database
- **TRUNCATE** - remove all records from a table, including all spaces allocated for the records are removed
- **COMMENT** - add comments to the data dictionary
- **RENAME** - rename an object

Data Manipulation Language (DML)

It is a language that provides a set of operations to support the basic data manipulation operations on the data held in the databases. It allows users to insert, update, delete and retrieve data from the database. The part of DML that involves data retrieval is called a query language. The following are an overview about the usage of DML statements in SQL:

- **SELECT** - retrieve data from the a database
- **INSERT** - insert data into a table
- **UPDATE** - updates existing data within a table
- **DELETE** - deletes all records from a table, the space for the records remain

Data Control Language (DCL)

DCL statements control access to data and the database using statements such as GRANT and REVOKE. A privilege can either be granted to a User with the help of GRANT statement. The privileges assigned can be SELECT, ALTER, DELETE, EXECUTE, INSERT, INDEX etc. In addition to granting of privileges, you can also revoke (taken back) it by using REVOKE command.

The following t are an overview about the usage of DCL statements in SQL:

- **GRANT** - gives user's access privileges to database
- **REVOKE** - withdraw access privileges given with the GRANT command

In practice, the data definition and data manipulation languages are not two separate languages. Instead they simply form parts of a single database language such as Structured Query Language (SQL). SQL represents combination of DDL and DML, as well as statements for constraints specification and schema evaluation.

Transaction Control Language (TCL)

A Transaction Control Language (TCL) is a Computer Language and a subset of SQL, used to control transactional processing in a database. A transaction is logical unit of work that comprises one or more SQL statements, usually a group of Data Manipulation Language (DML) statements.

Examples of TCL commands include:

- **COMMIT :** to apply the transaction by saving the database changes.
- **ROLLBACK :** to undo all changes of a transaction.
- **SAVEPOINT :** to divide the transaction into smaller sections. It defines breakpoints for a transaction to allow partial rollbacks.

Interfaces in DBMS

A Database Management System (DBMS) interface is a user-friendly platform that allows users and applications to interact with a database without needing in-depth knowledge of the underlying query languages.

- Interfaces in DBMS provide a way for users to interact with the database system
- They help users store, retrieve, update, and manage data easily
- Used to simplify data access, querying, and data manipulation

Types of Database Interfaces

1. Menu-Based Interfaces

Present users with lists of options (menus) that guide them through query formation step by step. Users select commands or data fields from these menus, eliminating the need to memorize complex query syntax.

- Reduces errors and simplifies navigation.

- Used in browsing interfaces, which allow a user look through the contents of database.
- **Example:** Library systems, E-commerce sites.

2. Forms-Based Interfaces

Display a predefined form that users fill to insert or retrieve data. Forms are designed and programmed for naive users as interfaces.

- Designed for users with minimal technical expertise.
- Ensures data consistency and validation by restricting input to specific fields, formats, and allowed values.
- **Example:** SQL forms, Student portals.

3. Graphical User Interfaces (GUI)

GUIs display the database schema to the user in a diagrammatic form. Users can create queries by interacting with the visual representation using a mouse or pointing device.

- Intuitive, visual and reduces the learning curve for complex databases.
- Many GUIs combine menus and forms for better usability.
- **Example:** MySQL Workbench

4. Natural Language Interfaces

Allow users to submit requests written in English or other human languages and attempt to understand them. A natural language interface usually has its own schema, which is similar to the database conceptual schema.

- User-friendly for non-technical users.
- Limited capabilities and may require clarification dialogues for ambiguous queries.
- **Example:** Chat-80 ,Modern Text-to-SQL tools.

5. Speech Input and Output Interfaces

Speech-based interfaces allow users to speak queries and receive results verbally. Applications with limited vocabulary, such as flight information, bank accounts or telephone directories.

- Enables access for users who cannot use keyboards.
- Speech input is converted into parameters for queries; output is converted from text/numbers into speech.
- **Example:** Google Assistant/Siri with CRM, Custom python assistants.

6. Interfaces for Parametric Users

Provide predefined commands that require minimal input and are used for repetitive operations, such as bank transactions or routine database updates. They are fast, efficient and reduces input errors.

- They target users who execute the same few operations daily.
- Provide limited flexibility and emphasize speed, accuracy, and consistency in routine operations
- **Example:** Bank teller systems, ATM interfaces.

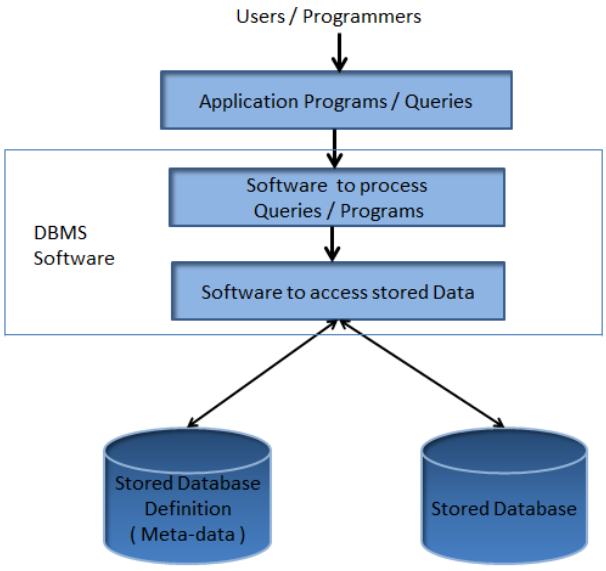
7. Interfaces for Database Administrators (DBA)

DBA interfaces provide privileged commands for managing the database system. These include:

- Creating and managing user accounts.
- Setting system parameters.
- Granting authorizations.
- Modifying database schemas.
- Reorganizing storage structures.
- **Example:** SQL Server Management Studio (SSMS), Oracle Enterprise Manager.

Database system environment

A database environment is a collective system of components that comprise and regulates the group of data, management, and use of data, which consist of software, hardware, people, techniques of handling database, and the data also.



Database System Environment

Here, the hardware in a database environment means the computers and computer peripherals that are being used to manage a database, and the software means the whole thing right from the operating system (OS) to the application programs that include database management software like M.S. Access or SQL Server.

Again the people in a database environment include those people who administrate and use the system. The techniques are the rules, concepts, and instructions given to both the people and the software along with the data with the group of facts and information positioned within the database environment.

A complete database environment is comprised of five major, interacting components:

- **Hardware:** The physical electronic devices required to run the system. This includes computers (servers, mainframes, workstations), storage devices (hard drives, SSDs, SANs), and network devices required to distribute data.
- **Software:** The suite of programs used to control and manage the overall database. This includes:

  - *The DBMS Software:* The core engine (e.g., MySQL, Oracle, PostgreSQL) that processes data access requests.
  - *Operating System (OS):* The underlying system software that runs the hardware.
  - *Application Programs & Utilities:* Tools and software interfaces used to access, analyze, and manipulate the data.

- **Data:** The most critical resource in the environment. Data acts as the bridge between the machine components (hardware/software) and the human components. It includes both the actual operational facts and the **metadata** (data about data, stored in a data dictionary).
- **Procedures:** The instructions and rules that govern the design and use of the database system. Procedures define how to log in, how to back up data, how to handle system failures, and how reports are generated.
- **People:** The individuals who interact with the system. This encompasses several roles:

  - *Database Administrators (DBAs):* Responsible for managing, securing, and maintaining the database environment.
  - *Database Designers:* Responsible for defining the data structure, relations, and constraints.
  - *Application Programmers:* Developers who write programs to interact with the database.
  - *End Users:* The people who ultimately need the data to perform their daily work (e.g., accountants, managers, customers).

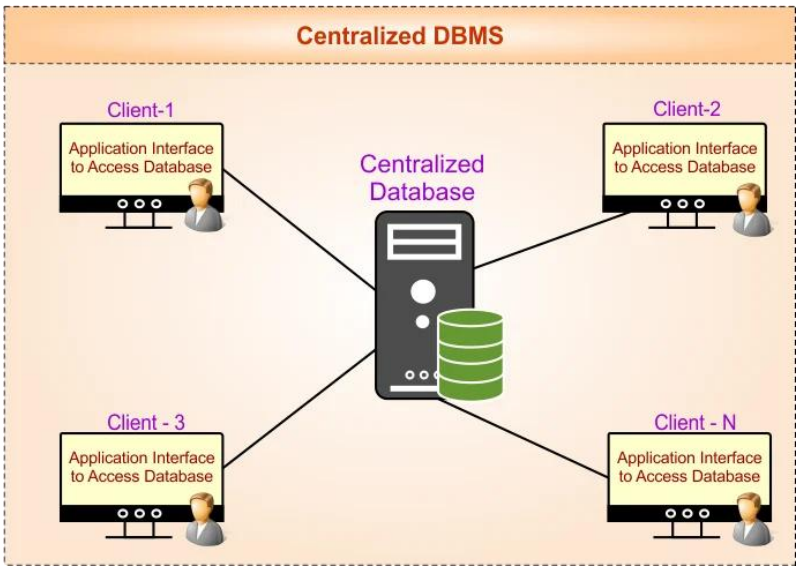
Centralized and Client/Server Architectures for DBMSs

The architecture of a Database Management System (DBMS) significantly influences its performance, scalability, and overall functionality.

Two primary architectures used for DBMSs are Centralized Architecture and Client/Server Architecture.

1. Centralized Architecture

- In a centralized DBMS architecture, the entire database system—including the database itself and the DBMS software—resides on a single central server.



Key Characteristics

- All users access the system through dumb terminals or remote user interfaces connected to this central machine
- The central server is responsible for handling all data storage, query processing, and transaction management
- Mainframe computers are used for processing all system functions including:
  - User application programs
  - User interface programs

Advantages

- Simplicity: Easy to understand and implement
- Consistent performance: All processing happens on one machine
- Easier backup and recovery: Single point of data management
- Better security: Centralized control over data access

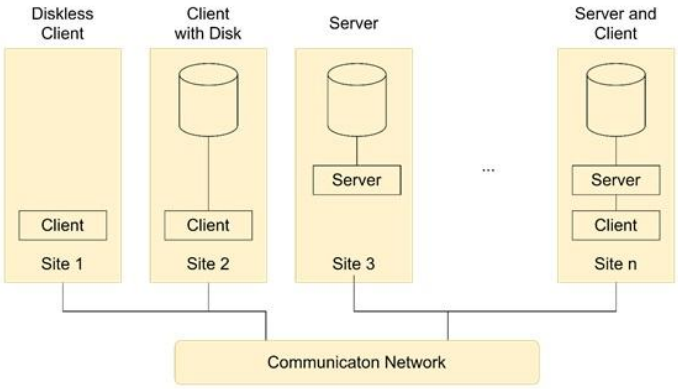
Disadvantages

- Scalability issues: Difficult to add more users
- Single point of failure: If the mainframe fails, entire system stops
- Performance bottlenecks: All users compete for same resources

- Limited flexibility: Terminals have no processing capability

2. **Client/Server Architecture**
    - In a client/server DBMS architecture, the database system is divided into two logical components:

The Server: Stores and manages the database.
The Client: Provides the user interface and sends requests to the server.



Key Characteristics

- Framework consists of many PCs and smaller number of mainframe machines connected via LANs and computer networks
- Client: User machine providing user interface capabilities and local processing
- Server: System containing hardware and software providing services (file access, printing, database access)
- Communication occurs over a network using standard protocols

Advantages

- Scalability: Easy to add more clients
- Distributed load: Processing shared between client and server
- Improved availability: Server can remain up even if clients fail
- Flexibility: Clients have local processing power

Disadvantages

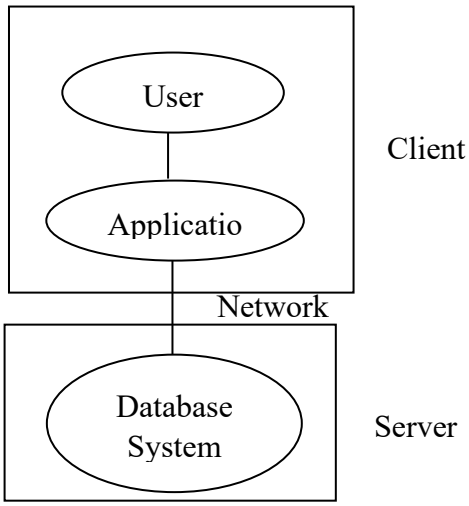
- Complexity: More difficult to design and maintain
- Network dependency: Requires reliable network connection
- Security challenges: Data transmitted over network
- Data consistency issues: More difficult to maintain

Aspect	Centralized Architecture	Client/Server Architecture
Processing Location	All on mainframe	Distributed between client and server
Terminals	Dumb terminals (no processing)	Smart clients (local processing)
Scalability	Poor	Good
Security	Better (centralized)	More challenging
Complexity	Simple	Complex
Failure Point	Single point of failure	More resilient
Network Dependency	Low	High
Maintenance	Easier backup/recovery	More complex

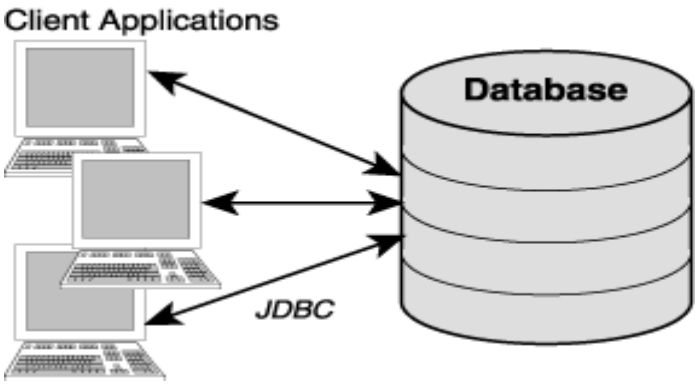
Client/Server Architecture Variants

i) **Two-tire Architecture:-**. The application is partitioned into a component that resides at the client machines which invokes database system functionally at the server machine through query language statement.

Application program interface like ODBC (open data based connectivity) and JDBC (Java data base connectivity) are used for interaction between client and the server.



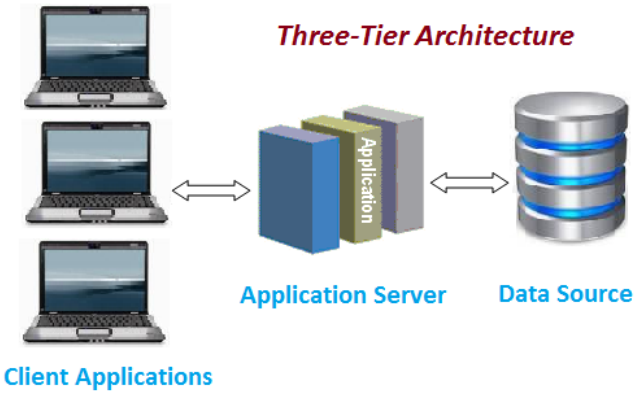
2-tire architecture



ii) **Three – tire Architecture:-** The client machine acts as a merily a font end and does not contain any direct data base calls instead the client in communication with an application server, usually through a form interface.

The application server in win communicates with data base system to access data.

Three type applications are more appropriate for large application for large application and for application that runs on would wave.



Classification of Database Management Systems

Database Management Systems (DBMS) can be classified along several dimensions based on their data model, architecture, user count, and usage context.

1. By Data Model

- **Relational DBMS (RDBMS)** Organizes data into tables (relations) with rows and columns. Uses SQL for querying. Examples: MySQL, PostgreSQL, Oracle, Microsoft SQL Server.
- **Hierarchical DBMS** Data is organized in a tree-like structure with parent-child relationships. Fast for predefined queries but rigid. Examples: IBM IMS, Windows Registry.
- **Network DBMS** An extension of hierarchical models allowing many-to-many relationships via a graph structure. Examples: IDS, IDMS.
- **Object-Oriented DBMS (OODBMS)** Stores data as objects (as in OOP), supporting complex data types. Examples: ObjectDB, db4o.
- **Object-Relational DBMS (ORDBMS)** Combines relational and object-oriented features. Examples: PostgreSQL (with OO extensions), Oracle.
- **NoSQL DBMS** Designed for unstructured or semi-structured data. Includes four sub-types:
    - **Document stores** — MongoDB, CouchDB
    - **Key-value stores** — Redis, DynamoDB
    - **Column-family stores** — Apache Cassandra, HBase
    - **Graph databases** — Neo4j, Amazon Neptune
- **NewSQL DBMS** Modern relational systems designed for scalability like NoSQL while maintaining ACID compliance. Examples: CockroachDB, Google Spanner.

2. By Number of Users

- Single-user: Supports one user at a time; typical for desktops. Eg. Microsoft Access, SQLite
- Multi-user: Supports concurrent access by multiple users Eg. Oracle, PostgreSQL, MySQL

3. By Distribution

- **Centralized DBMS** — All data and processing reside on a single server/location.
- **Distributed DBMS (DDBMS)** — Data is distributed across multiple physical locations but appears as one logical database. Can be homogeneous or heterogeneous.
- **Cloud DBMS** — Hosted and managed on cloud infrastructure, offering elasticity and managed services. Examples: Amazon RDS, Google Cloud Spanner, Azure SQL.